

NATIONAL TECHNICAL UNIVERSITY OF UKRAINE “IGOR SIKORSKY
KYIV POLYTECHNIC INSTITUTE” EDUCATIONAL AND SCIENTIFIC
INSTITUTE OF ATOMIC AND THERMAL POWER ENGINEERING
DEPARTMENT OF DIGITAL TECHNOLOGIES IN POWER ENGINEERING

CALCULATED AND GRAPHICS WORK

In the discipline “Visualization of graphical and geometric information”

Topic: “Operations on texture coordinates”

Variant 5

Completed by:

Student of group TR-51mp

Bodnar Orest Serhiiovych

Kyiv – 2025

1. Task Description

1.1 Assignment Overview

This assignment focuses on implementing advanced texture coordinate operations for parametric surfaces. Building upon the foundation established in Practical Assignment 3 (texture mapping with diffuse, normal, and specular maps), this work extends the functionality by introducing dynamic texture rotation around a user-specified pivot point.

1.2 Requirements

The assignment requirements are specified as follows:

1. Reuse Previous Work: the implementation must reuse the texture mapping system from the Control task (PA#3), which includes multi-component texture support (diffuse, specular, and normal maps).
2. Texture Rotation: implement texture coordinate rotation around a user-specified point. The rotation must be applied to all texture components uniformly while maintaining proper normal map orientation.
3. Interactive Pivot Point Control: the rotation center point must be adjustable in the surface's (r,v) parametric space using keyboard controls:
 - Keys W and S move the point along the r parameter
 - Keys A and D move the point along the v parameter
4. Real-time Interaction: all operations must be performed in real-time with immediate visual feedback.

2. Theoretical Background

2.1 Geometric Foundation: Surface of Revolution

The parametric surface used in this implementation is a Surface of Revolution Σ with wave deformation. A surface of revolution is generated by revolving a 2D profile curve about an axis. The parametric representation of a point p on the surface is given by:

$$p(r, u) = \begin{pmatrix} r \cos(u) \\ r \sin(u) \\ z(r) \end{pmatrix}$$

where r is the radial parameter, $u \in [0, 2\pi)$ is the angular parameter, and $z(r)$ represents the height function along the axis of revolution. In this implementation, the height function incorporates a damped wave pattern:

$$z(r) = a \cdot e^{-n \cdot r} \cdot \sin(w \cdot r + f)$$

where a is the amplitude, n is the damping coefficient, w is the angular frequency, and f is the phase offset.

2.2 Texture Coordinate System

The texture coordinates (u, v) define a mapping from the 3D surface to the 2D texture space, typically normalized to $[0, 1] \times [0, 1]$. For the surface of revolution, the natural mapping is:

$$u_{tex} = \frac{u}{2\pi}, v_{tex} = \frac{r - r_{min}}{r_{max} - r_{min}}$$

This mapping ensures that the texture wraps continuously around the angular direction and spans uniformly along the radial direction.

2.3 Texture Rotation Transformation

Texture rotation involves transforming the 2D texture coordinates before sampling. To rotate a texture by an angle θ around a pivot point $P_{pivot}=(u_p, v_p)$, a three-step transformation is applied:

1. Translation to Origin:

$$T_{to_origin}(u, v) = (u - u_p, v - v_p)$$

2. Rotation: apply the 2D rotation matrix $R(\theta)$:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

3. Translation back:

$$T_{back}(u', v') = (u' + u_p, v' + v_p)$$

The complete transformation is expressed as:

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = T_{back} \left(R(\theta) \cdot T_{to_origin} \begin{pmatrix} u \\ v \end{pmatrix} \right)$$

Expanding this yields:

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} \cos \theta \cdot (u - u_p) - \sin \theta \cdot (v - v_p) + u_p \\ \sin \theta \cdot (u - u_p) + \cos \theta \cdot (v - v_p) + v_p \end{pmatrix}$$

2.4 Tangent Space and Normal Mapping

The accurate rendering of normal-mapped surfaces requires a proper tangent space basis at each vertex. This basis consists of three vectors:

- Normal (N): perpendicular to the surface
- Tangent (T): aligned with the u texture coordinate direction
- Bitangent (B): aligned with the v texture coordinate direction

The tangent and bitangent are computed from the parametric partial derivatives:

$$T = \frac{\partial p}{\partial u}, B = \frac{\partial p}{\partial r}$$

These vectors, together with the surface normal, form the tangent space coordinate system necessary for correctly transforming normal map data from texture space to world space for lighting calculations.

2.5 Illumination Model

The Phong illumination model is employed for shading, incorporating contributions from ambient, diffuse, and specular components. The final fragment color I is computed as:

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$

where:

- Ambient: $I_{ambient} = k_a \cdot C_{diffuse}(u', v')$
- Diffuse: $I_{diffuse} = k_d \cdot C_{diffuse}(u', v') \cdot (N_{perturbed} \cdot L)$
- Specular: $I_{specular} = k_s \cdot C_{specular}(u', v') \cdot (R \cdot V)^{shininess}$

Here, $C_{diffuse}(u', v')$ and $C_{specular}(u', v')$ are sampled from the rotated texture coordinates, $N_{perturbed}$ is the normal perturbed by the normal map, L is the light direction, R is the reflection vector, and V is the view direction.

3. Implementation Details

3.1 System Architecture

The implementation follows a modular architecture with clear separation of concerns:

- Application Layer (Application.js): manages application lifecycle, user input, and coordination between components
- Rendering Layer (Renderer.js): handles WebGL state management, uniform updates, and rendering pipeline
- Model Layer (ParametricSurface.js, BaseModel.js): generates and manages surface geometry, normals, and tangent space
- Mathematics Layer (SurfaceMath.js): provides mathematical utilities for surface generation and vector operations
- Shader Layer (vertexShader.glsl, fragmentShader.glsl): implements GPU-side transformations and lighting calculations

3.2 Shader Implementation

3.2.1 Vertex Shader

The vertex shader performs the texture coordinate rotation transformation. Key additions include:

```
uniform float textureRotationAngle;
uniform vec2 rotationCenter;
```

The rotation is applied to the texture coordinates before passing them to the fragment shader:

```
vec2 translatedCoord = texCoord - rotationCenter;
float cosAngle = cos(textureRotationAngle);
float sinAngle = sin(textureRotationAngle);
vec2 rotatedCoord = vec2(
    cosAngle * translatedCoord.x - sinAngle * translatedCoord.y,
    sinAngle * translatedCoord.x + cosAngle * translatedCoord.y
);
vTexCoord = rotatedCoord + rotationCenter;
```

This implementation efficiently computes the rotation on the GPU for each vertex, with the transformed coordinates being interpolated across fragments.

3.2.2 Fragment Shader

The fragment shader samples all texture maps using the rotated coordinates and computes the final shaded color. An additional feature is the visual rotation center indicator:

```
if (showRotationCenter) {  
    float dotRadius = 0.02;  
    float distance = length(vTexCoord - rotationCenter);  
    if (distance < dotRadius) {  
        float intensity = smoothstep(dotRadius, dotRadius * 0.5,  
distance);  
        vec3 dotColor = vec3(1.0, 0.0, 0.0);  
        finalColor = mix(finalColor, dotColor, intensity);  
    }  
}
```

This creates a smooth, anti-aliased red dot at the rotation center for visual feedback.

3.3 Tangent Space Computation

The tangent space basis is required for normal mapping. The implementation computes:

Normal Calculation: vertex normals are computed using angle-weighted averaging of adjacent face normals, providing smooth interpolation across the mesh.

Tangent and Bitangent: tangent and bitangent vectors are derived from the parametric surface derivatives to form the tangent space coordinate system at each vertex.

3.4 Keyboard Input Handling

Keyboard controls are implemented through an event listener that updates the rotation center position:

```
setupKeyboardControls() {  
  const moveStep = 0.01;  
  document.addEventListener('keydown', (event) => {  
    let newR = currentR;  
    let newV = currentV;  
    switch(event.key.toLowerCase()) {  
      case 'w': newR = Math.min(1, currentR + moveStep);  
break;  
      case 's': newR = Math.max(0, currentR - moveStep);  
break;  
      case 'a': newV = Math.max(0, currentV - moveStep);  
break;  
      case 'd': newV = Math.min(1, currentV + moveStep);  
break;  
    }  
    this.renderer.setRotationCenter(newR, newV);  
    // Update UI sliders and redraw  
  });  
}
```

The implementation ensures that: - Movement is clamped to the valid [0,1] range - UI sliders are synchronized with keyboard input - The scene is immediately redrawn to show changes

3.5 User Interface Design

The user interface employs a side-by-side layout with the WebGL canvas on the left and control panels on the right. The controls are organized into two groups:

1. Texture Rotation Controls:

- Rotation angle slider (0-360°)
- Rotation center R coordinate slider (0-1)
- Rotation center V coordinate slider (0-1)

- Checkbox to toggle rotation center visualization
- Keyboard control instructions

2. Mesh Density Controls:

- R steps slider (radial subdivision)
- V steps slider (angular subdivision)

CSS styling provides visual hierarchy and clear grouping of related controls.

4. User Instructions

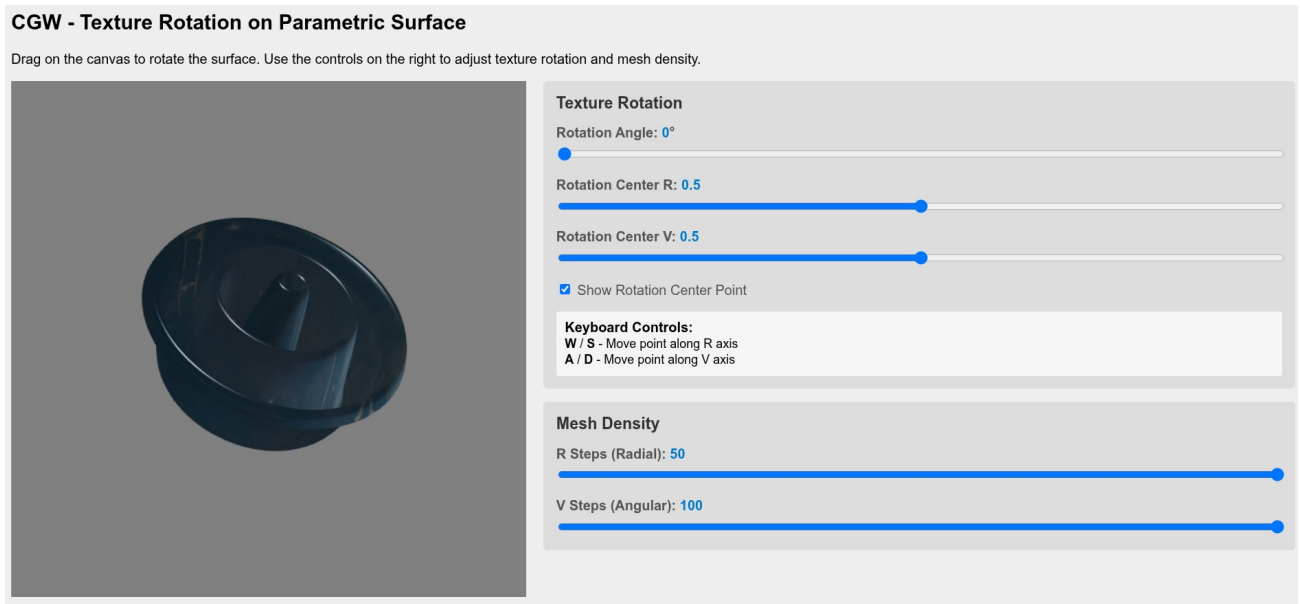
4.1 Initial Setup

To run the application:

1. Ensure all files are in the correct directory structure:

```
WebGL/
├── index.html
├── Utils/
│   ├── m4.js
│   └── trackball-rotator.js
├── src/
│   ├── Application.js
│   ├── config/surfaceConfig.js
│   ├── core/
│   │   ├── Renderer.js
│   │   ├── SurfaceMath.js
│   │   └── TextureManager.js
│   ├── models/
│   │   ├── BaseModel.js
│   │   └── ParametricSurface.js
│   └── shaders/
│       ├── vertexShader.glsl
│       ├── fragmentShader.glsl
│       └── ShaderLoader.js
└── textures/
    ├── diffuse.jpg
    ├── specular.jpg
    └── normal.jpg
```

2. Open index.html in a WebGL-compatible browser (Chrome, Firefox, Edge, or Safari).
3. The application will load and display the parametric surface with default settings.



4.2 Camera Controls

Mouse Interaction: - Click and Drag on the canvas to rotate the camera around the surface - The trackball rotation system provides intuitive 3D viewing control

4.3 Texture Rotation Controls

Rotation Angle:

- Use the “Rotation Angle” slider to rotate the texture from 0° to 360°
- The displayed value updates in real-time
- All texture maps (diffuse, specular, normal) rotate simultaneously

Rotation Center Position:

- Use the “Rotation Center R” slider to move the pivot point along the radial axis (0.0 to 1.0)

- Use the “Rotation Center V” slider to move the pivot point along the angular axis (0.0 to 1.0)

- A red dot indicates the rotation center when visualization is enabled

Keyboard Controls (Alternative to Sliders):

- Press W to move the rotation center up (increase R)

- Press S to move the rotation center down (decrease R)

- Press A to move the rotation center left (decrease V)

- Press D to move the rotation center right (increase V) - Movement step size is 0.01 per key press

4.4 Mesh Density Controls

Mesh Refinement:

R Steps (Radial) controls subdivision along the radial direction (3-50 steps), higher values create smoother radial profiles.

V Steps (Angular) controls subdivision around the circumference (3-100 steps), higher values create smoother circular cross-sections.

5. Source Code Samples

5.1 Vertex Shader (Texture Rotation)

```
attribute vec3 vertex;
attribute vec3 normal;
attribute vec2 texCoord;
attribute vec3 tangent;
attribute vec3 bitangent;

uniform mat4 ModelViewProjectionMatrix;
uniform mat4 ModelViewMatrix;
uniform mat4 NormalMatrix;

uniform float textureRotationAngle;
uniform vec2 rotationCenter;

varying vec3 vNormal;
varying vec3 vPosition;
```

```

varying vec2 vTexCoord;
varying vec3 vTangent;
varying vec3 vBitangent;

void main() {
    vec4 position = ModelViewMatrix * vec4(vertex, 1.0);
    vPosition = position.xyz;

    // Transform normal, tangent, and bitangent to view space
    vNormal = normalize((NormalMatrix * vec4(normal, 0.0)).xyz);
    vTangent = normalize((NormalMatrix * vec4(tangent, 0.0)).xyz);
    vBitangent = normalize((NormalMatrix * vec4(bitangent, 0.0)).xyz);

    // Apply texture rotation around user-specified point
    vec2 translatedCoord = texCoord - rotationCenter;
    float cosAngle = cos(textureRotationAngle);
    float sinAngle = sin(textureRotationAngle);
    vec2 rotatedCoord = vec2(
        cosAngle * translatedCoord.x - sinAngle * translatedCoord.y,
        sinAngle * translatedCoord.x + cosAngle * translatedCoord.y
    );
    vTexCoord = rotatedCoord + rotationCenter;

    gl_Position = ModelViewProjectionMatrix * vec4(vertex, 1.0);
}

```

5.2 Tangent Space Generation

```

/**
 * Generate tangent space vectors for normal mapping
 */
generateTangentSpace(normalsArray) {
    const tangents = [];
    const bitangents = [];
    const { rRange, uRange } = this.config;

    let normalIndex = 0;
    for (let i = 0; i <= this.rSteps; i++) {
        const r = rRange.min + (rRange.max - rRange.min) * (i /
this.rSteps);
        for (let j = 0; j <= this.uSteps; j++) {
            const u = uRange.min + (uRange.max - uRange.min) * (j /
this.uSteps);

            // Compute tangent and bitangent vectors from surface
            derivatives
            const tangent = SurfaceMath.surfaceTangent(r, u);
            const bitangent = SurfaceMath.surfaceBitangent(r, u,
this.config);

```

```

        tangents.push(tangent.x, tangent.y, tangent.z);
        bitangents.push(bitangent.x, bitangent.y, bitangent.z);

        normalIndex++;
    }
}

return {
    tangents: new Float32Array(tangents),
    bitangents: new Float32Array(bitangents)
};
}

```

5.3 Keyboard Input Handler

```

/**
 * Set up keyboard controls for moving rotation center
 */
setupKeyboardControls() {
    const moveStep = 0.01; // Movement step size

    document.addEventListener('keydown', (event) => {
        let moved = false;
        const currentR = this.renderer.rotationCenter[0];
        const currentV = this.renderer.rotationCenter[1];
        let newR = currentR;
        let newV = currentV;

        switch(event.key.toLowerCase()) {
            case 'a':
                newV = Math.max(0, currentV - moveStep);
                moved = true;
                break;
            case 'd':
                newV = Math.min(1, currentV + moveStep);
                moved = true;
                break;
            case 'w':
                newR = Math.min(1, currentR + moveStep);
                moved = true;
                break;
            case 's':
                newR = Math.max(0, currentR - moveStep);
                moved = true;
                break;
        }
    })
}

```

```

    if (moved) {
        event.preventDefault();
        this.renderer.setRotationCenter(newR, newV);

        // Update UI sliders to reflect keyboard input
        const rotationCenterRSlider =
            document.getElementById("rotationCenterRSlider");
        const rotationCenterVSlider =
            document.getElementById("rotationCenterVSlider");
        const rotationCenterRValue =
            document.getElementById("rotationCenterRValue");
        const rotationCenterVValue =
            document.getElementById("rotationCenterVValue");

        if (rotationCenterRSlider && rotationCenterRValue) {
            rotationCenterRSlider.value = newR * 100;
            rotationCenterRValue.textContent = newR.toFixed(2);
        }

        if (rotationCenterVSlider && rotationCenterVValue) {
            rotationCenterVSlider.value = newV * 100;
            rotationCenterVValue.textContent = newV.toFixed(2);
        }

        this.draw();
    }
});
}

```

5.4 Fragment Shader (Rotation Center Visualization)

```

// Draw rotation center point if enabled
if (showRotationCenter) {
    float dotRadius = 0.02;
    float distance = length(vTexCoord - rotationCenter);

    if (distance < dotRadius) {
        // Create a smooth circular dot with bright red color
        float intensity = smoothstep(dotRadius, dotRadius * 0.5,
distance);
        vec3 dotColor = vec3(1.0, 0.0, 0.0);
        finalColor = mix(finalColor, dotColor, intensity);
    }
}

```

5.5 Renderer Uniform Management

```

/**
 * Set shader uniforms
 */
setUniforms(matrices, lightPosition) {
  const { shaderProgram: sp } = this;

  // Matrices
  this.gl.uniformMatrix4fv(sp.iModelViewProjectionMatrix,
    false, matrices.modelViewProjection);
  this.gl.uniformMatrix4fv(sp.iModelViewMatrix,
    false, matrices.modelView);
  this.gl.uniformMatrix4fv(sp.iNormalMatrix,
    false, matrices.normalMatrix);

  // Light
  this.gl.uniform3fv(sp.iLightPosition, lightPosition);

  // Material properties
  this.gl.uniform3fv(sp.iAmbientColor, [0.6, 0.6, 0.6]);
  this.gl.uniform3fv(sp.iDiffuseColor, [1.5, 1.5, 1.5]);
  this.gl.uniform3fv(sp.iSpecularColor, [1.0, 1.0, 1.0]);
  this.gl.uniform1f(sp.iShininess, 16.0);

  // Texture rotation
  this.gl.uniform1f(sp.iTextureRotationAngle,
    this.textureRotationAngle);
  this.gl.uniform2fv(sp.iRotationCenter, this.rotationCenter);
  this.gl.uniform1i(sp.iShowRotationCenter,
    this.showRotationCenter ? 1 : 0);
}

```